CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Yunus Dalkıran
Öğrenci numarası (SID)	5001240017
E-posta	yunusresmimail@gmail.com
Seçilen görev (G1–G5)	G2 - Sınav notu dağılımı
Seçilen ISA ve sürümü (ör. RV32IM)	RV32IM (32 bit)
Derleyici ve sürümü	riscv64-elf-gcc 16.2.0
Simülatör / araçlar	Ripes 2.2.6
Git deposu adresi	github.com/ yunD-19/ Computer_organization

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	Son hane = 7	7
D2	SID'in son iki hanesi	Son iki hane = 17	17
S	SID'in hanelerinin toplamı	5+0+0+1+2+4+0+0+1+7	20
N	24 + (D2 mod 25)	24+(17 mod 25)	41 eleman
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d=7 tablosu	128 B; 32 B line direct; 4 line/set

Write policy	d çift → write-back, d tek → write-through	d=7 tek	write-through
k	(S mod 7) + 3	(20 mod 7)+3	9. komut
Interrupt türü	S çift → Timer, S tek → I/O	S=20 çift	Timer
f	1.0 + 0.1 × d GHz (varsayım)	1,0+0,1x7	1,7 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1=1; T2=20+5x3	1 / 35 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2+(20 mod 7)	8 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	50+(1240017 mod 10)	P=57 puan

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	5 bit	log2(32 byte)
Line (r) veya set (d)	2 bit	log2(4 line)
Tag (s − r veya s − d)	25 bit	32-5-2
Toplam	32 bit	25+2+5

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☒ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

Bkz. Ek A (s. 16): öğrencinin onayladığı 5 cümle.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

Bkz. Ek B (s. 17): satır numaralı C kodu, 52 satır.

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	2,16-20,22-51	Global dizi; iki geçiş
Y2	Koşullu dallanma	23,28,45	if koşulları
Y3	Parametreli, değer döndüren fonksiyon çağrısı	8-10,17	next_x çağrısı
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	9,18,47	LCG; shift; OR

Y5	Dizi elemanına store	18,47	Diziye yazma
Y6	const int SID ve hesapta kullanımı	1,14	SID ile LCG başlangıcı
Y7	Sonucun tutulduğu global değişken(ler)	3-6,21,24,30-42,46	Global sonuçlar

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
mean	0x00013564	55	ripes_global_counts.png
below_count	0x00013568	19	ripes_global_counts.png
pass_count	0x0001356c	22	ripes_global_counts.png
bucket[0..4]	0x13668..0x13678	18,3,7,2,11	ripes_bucket_counts.png

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	16-20
Başlangıç ve bitiş adresi (hex)	0x102cc -> 0x1028c
Dinamik instruction sayısı	20
Hangi döngü iterasyonu (ör. i = 3)	i=0 -> i=1 çağrısı

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
14	x = SID	79-81	0x10270-0x10278
15	gradeSum = 0	82	0x1027c
17	x = next_x(x)	86-88	0x10288-0x10290
18	notu diziye yaz	89-107	0x10294-0x102d8
19	gradeSum += not	108-116	0x102dc-0x102f8
21	mean = toplam/41	124-132	0x10314-0x10330

Tam esleme: Ek C (s. 18).

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x000100b4	10172 byte	kod + runtime

.data / .bss	0x13010 / 0x13560 / 0x135ac	1336 / 76 / 960 byte	SID ve global veri

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

> Komutlar .text, veriler .data/.bss bölümlerindedir; ikisi aynı adres uzayındadır.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	20	4	Çağrı gömüldü; Ek D, s. 19
Load sayısı	7	0	Değerler register içinde
Store sayısı	3	1	Stack yazımları azaldı
Kullanılan register sayısı	7	4	Yalnız bu iz penceresinde

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x10270	lui a5,0x12f	U	Ek E, #1	0x0012f7b7	b7 f7 12 00	Aynı
2	0x10274	addi a5,a5,-1071	I	Ek E, #2	0xbd178793	93 87 17 bd	Aynı
3	0x10288	lw a0,-20(s0)	I	Ek E, #3	0xfec42503	03 25 c4 fe	Aynı
4	0x1028c	jal 0x10214	J	Ek E, #4	0xf89ff0ef	ef f0 9f f8	Aynı
5	0x102a4	mulhu a4,a5,a4	R	Ek E, #5	0x02e7b733	33 b7 e7 02	Aynı
6	0x102d0	slli a5,a5,2	I	Ek E, #6	0x00279793	93 97 27 00	Aynı
7	0x102d4	add a5,a4,a5	R	Ek E, #7	0x00f707b3	b3 07 f7 00	Aynı
8	0x102d8	sw a3,0(a5)	S	Ek E, #8	0x00d7a023	23 a0 d7 00	Aynı
9	0x10300	addi a5,a5,1	I	Ek E, #9	0x00178793	93 87 17 00	Aynı
10	0x10310	bge a5,a4,0x10288	B	Ek E, #10	0xf6e7dce3	e3 dc e7 f6	Aynı

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 1-2
- ☒ Koşullu branch: # 10
- ☒ Load: # 3
- ☒ Store: # 8
- ☒ Fonksiyon çağrısı: # 4

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

> LUI register'a alt 12 biti 0 olan 0x12f000 değerini koyar; ADDI tek başına SID'i taşıyamaz.
> SID'in alt 12 biti 0xbd1, işaretli 12 bit olarak -1071'dir; üst bölüm 0x12e yerine 0x12f seçilir.
> ADDI -1071 ekleyince register'da 0x12ebd1, yani 1240017 elde edilir.

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

> bge, i < 41 iken döngünün sürmesini sağlar. Komut adresi 0x10310, hedef 0x10288'dir.
> Hedef - komut = 0x10288 - 0x10310 = -0x88 = -136 byte; koşul sağlanınca 136 byte geriye gider.

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

>

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										

16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 `ADD B,A` ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			

MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	

Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			

Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		

T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	Ic × CPI × τ		
MIPS rate	f / (CPI × 10^6)		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	Ic × [p + (m × k)] × τ		
T, cache var (k = T_s)	Ic × [p + (m × k)] × τ		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	

Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = 1 / [(1 − f) + f / Nc]	
Nc → ∞ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu

- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.
M2 kullanım beyanı ve doğrulama: Ek G (s. 22).

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Codex (GPT-6)	0-1	Kavram, hata ayıklama	LCG formülü ne?	GCC, kod, yönerge
Codex (GPT-6)	M1	Kernel ve sonuç kontrolü	Bu kernel uygun mu?	objdump, Ripes
Codex (GPT-6)	M1	Dil ve PDF düzenleme	M1 taslağını kontrol et	Öğrenci incelemesi

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Yunus Dalkıran	5001240017		

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		

Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (`-O1` listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza

Ek A - M1 çözüm tasarımı

Yunus Dalkıran | 5001240017 | G2 | RV32IM

İlk adım olarak LCG formülü ile yeni x değerini üreten bir fonksiyon yazdım. İlk döngü içerisinde bu fonksiyonu çağırıp x değerlerinden notları hesapladım, notları global int array indexlerine koydum ve topladım; global diziyi Y1 şartı için seçtim, böylece notları ve işaretlemeleri Ripes belleğinde kontrol edebiliyorum. İlk döngü bittikten sonra toplamı 41’e bölerek mean değişkenini hesapladım. İkinci döngü içerisinde önce pass_count değişkenini güncelledim, daha sonra bucket arrayinin ilgili indexini notun aralığına göre artırdım ve ortalamanın altındaki notları below_count değişkeninde saydım. Daha sonra ortalamanın altında kalan notları OR 0x100 ile işaretledim ve döngüyü bitirdim.

ISA seçimi gerekçesi:

RV32IM seçme sebebim kodu GCC ile derleyip Ripes'te adım adım inceleyebilmek.

Ek B - Satır numaralı C kaynak kodu

Yunus Dalkıran | 5001240017 | G2 | 52 satır

```
01  const int SID = 1240017;
02  int studentGrades[41];
03  int bucket[5];
04  int mean;
05  int below_count;
06  int pass_count;
07
08  unsigned int next_x(unsigned int current_x){
09      unsigned int next_x = 1664525u * current_x + 1013904223u;
10      return next_x;
11  }
12
13  int main(){
14      unsigned int x = (unsigned int)SID;
15      int gradeSum = 0;
16      for (int i = 0;i<41;i++){
17          x = next_x(x);
18          studentGrades[i] = (x>>16) %101;
19          gradeSum += studentGrades[i];
20      }
21      mean = gradeSum/41;
22      for (int i = 0;i<41;i++){
23          if(studentGrades[i]>=57){
24              pass_count++;
25          }
26
27          int whichBucket;
28          if(studentGrades[i]<50){
29              whichBucket = 0;
30              bucket[whichBucket]++;
31          } else if(studentGrades[i]<65 && studentGrades[i]>49){
32              whichBucket = 1;
33              bucket[whichBucket]++;
34          } else if(studentGrades[i]<75 && studentGrades[i]>64){
35              whichBucket = 2;
36              bucket[whichBucket]++;
37          } else if(studentGrades[i]<85 && studentGrades[i]>74){
38              whichBucket = 3;
39              bucket[whichBucket]++;
40          } else if(studentGrades[i]<=100 && studentGrades[i]>84){
41              whichBucket=4;
42              bucket[whichBucket]++;
43          }
44
45          if(studentGrades[i]<mean){
46              below_count++;
47              studentGrades[i] = studentGrades[i] | 0x100;
48          }
49
50
51      }
52  }
```

Bu ek, resmi tutanağın 1.3 alanına bağlıdır.

Ek C - M2 / 2a C ve assembly eşlemesi

Kaynak: ckodu.c (52 satir); tam assembly: asm_O0.s; -O0, RV32IM.

C satiri	C islemi	asm_O0.s satiri
1	SID sabitinin tanimi	7-13
2	studentGrades[41] tanimi	14-20
3	bucket[5] tanimi	21-26
4	mean tanimi	27-33
5	below_count tanimi	34-39
6	pass_count tanimi	40-45
8	next_x fonksiyon girisi	47-55
9	LCG carpma ve toplama	56-63
10	sonucu dondurme	64-65
11	next_x fonksiyon cikisi	66-70
13	main girisi ve stack hazirligi	71-78
14	x = SID	79-81
15	gradeSum = 0	82
16	ilk for: baslangic, artirma, kosul	83-85, 117-123
17	next_x cagrisi ve x guncellemesi	86-88
18	not hesaplama ve diziye yazma	89-107
19	gradeSum artirma	108-116
21	mean = gradeSum / 41	124-132
22	ikinci for: baslangic, artirma, kosul	133-135, 332-339
23	not >= 57 kosulu	136-143
24	pass_count artirma	144-148
27	whichBucket tanimi; ayri komut yok	-
28	not < 50 kosulu	149-157
29	whichBucket = 0	158
30	bucket[whichBucket] artirma	159-172
31	50-64 araligi kosulu	173-189
32	whichBucket = 1	190-191
33	bucket[whichBucket] artirma	192-205
34	65-74 araligi kosulu	206-222
35	whichBucket = 2	223-224
36	bucket[whichBucket] artirma	225-238
37	75-84 araligi kosulu	239-255
38	whichBucket = 3	256-257
39	bucket[whichBucket] artirma	258-271
40	85-100 araligi kosulu	272-288
41	whichBucket = 4	289-290
42	bucket[whichBucket] artirma	291-303
45	not < mean kosulu	304-313
46	below_count artirma	314-318
47	OR 0x100 ile not isaretleme	319-331
52	main cikisi	340-346

Bos satirlar ve suslu parantezler icin ayri komut yoktur.

Dizi/degsiken bildirimleri assembler direktifi olabilir; instruction degildir.

Hex adres ornekleri tutanagin 2a tablosundadir; baglama sonrasi adresler ELF'e aittir.

Ek D - M2 / 2b kernel karşılaştırması

M1 izi: ilk döngüde i=0 için notu yazmaktan i=1 için next_x çağrısına kadar.

Aşağıdaki sıra dinamiktir; adres sırası değildir. -O1 aynı kaynak işlemlerin izidir.

-O0: 20 dinamik instruction

```
01  0x102cc  lw    a5,-28(s0)
02  0x102d0  slli  a5,a5,0x2
03  0x102d4  add   a5,a4,a5
04  0x102d8  sw    a3,0(a5)
05  0x102dc  addi  a4,gp,-588
06  0x102e0  lw    a5,-28(s0)
07  0x102e4  slli  a5,a5,0x2
08  0x102e8  add   a5,a4,a5
09  0x102ec  lw    a5,0(a5)
10  0x102f0  lw    a4,-24(s0)
11  0x102f4  add   a5,a4,a5
12  0x102f8  sw    a5,-24(s0)
13  0x102fc  lw    a5,-28(s0)
14  0x10300  addi  a5,a5,1
15  0x10304  sw    a5,-28(s0)
16  0x10308  lw    a4,-28(s0)
17  0x1030c  li    a5,40
18  0x10310  bge   a5,a4,10288
19  0x10288  lw    a0,-20(s0)
20  0x1028c  jal   10214
```

-O1: 4 dinamik instruction

```
01  0x102bc  sw    a5,0(a3)
02  0x102c0  add   a0,a0,a5
03  0x102c4  addi  a3,a3,4
04  0x102c8  bne   t5,a3,10294
```

Sayım: -O0 7 lw + 3 sw; -O1 0 lw + 1 sw. Register: -O0 7, -O1 4.

Register sayısı, pencerede okunan/yazılan farklı GPR sayısıdır; jal için ra dahildir.

-O0 sonunda jal ile next_x içine girilir. Fonksiyonun gövdesi 20 komutluk iz dışındadır.

-O1 fonksiyonu döngü içine gömer; x ve toplamı register içinde tutar, stack erişimini azaltır.

-O1 izi bne ile bir sonraki iterasyona geçer; sonraki mul iki izde de pencere dışındadır.

Bu sayılar tam döngü turu sayıları (54 / 14) değildir; M1 kernel penceresini karşılaştırır.

Doğrulama: objdump_O0.txt ve objdump_O1.txt; ELF dosyaları aynı depodadır.

Ek E - M2 / 2c elle kodlama alanları

Bit alanlarını elle hesapladım; 10/10 machine word objdump_O0.txt ile doğrulandı.

Alan sırası: U/I/R/S/B/J formatına göre; byte sırası RV32IM little-endian.

#1 0x10270 lui a5,0x12f (U)

```
Alanlar: 00000000000100101111 | 01111 | 0110111
32 bit: 0000 0000 0001 0010 1111 0111 1011 0111
Kod: 0x0012f7b7   Bellek: b7 f7 12 00   objdump: aynı
```

#2 0x10274 addi a5,a5,-1071 (I)

```
Alanlar: 101111010001 | 01111 | 000 | 01111 | 0010011
32 bit: 1011 1101 0001 0111 1000 0111 1001 0011
Kod: 0xbd178793   Bellek: 93 87 17 bd   objdump: aynı
```

#3 0x10288 lw a0,-20(s0) (I)

```
Alanlar: 111111101100 | 01000 | 010 | 01010 | 0000011
32 bit: 1111 1110 1100 0100 0010 0101 0000 0011
Kod: 0xfec42503   Bellek: 03 25 c4 fe   objdump: aynı
```

#4 0x1028c jal 0x10214 (J)

```
Alanlar: 1 | 1111000100 | 1 | 11111111 | 00001 | 1101111
32 bit: 1111 1000 1001 1111 1111 0000 1110 1111
Kod: 0xf89ff0ef   Bellek: ef f0 9f f8   objdump: aynı
```

#5 0x102a4 mulhu a4,a5,a4 (R)

```
Alanlar: 0000001 | 01110 | 01111 | 011 | 01110 | 0110011
32 bit: 0000 0010 1110 0111 1011 0111 0011 0011
Kod: 0x02e7b733   Bellek: 33 b7 e7 02   objdump: aynı
```

#6 0x102d0 slli a5,a5,2 (I)

```
Alanlar: 000000000010 | 01111 | 001 | 01111 | 0010011
32 bit: 0000 0000 0010 0111 1001 0111 1001 0011
Kod: 0x00279793   Bellek: 93 97 27 00   objdump: aynı
```

#7 0x102d4 add a5,a4,a5 (R)

```
Alanlar: 0000000 | 01111 | 01110 | 000 | 01111 | 0110011
32 bit: 0000 0000 1111 0111 0000 0111 1011 0011
Kod: 0x00f707b3   Bellek: b3 07 f7 00   objdump: aynı
```

#8 0x102d8 sw a3,0(a5) (S)

```
Alanlar: 0000000 | 01101 | 01111 | 010 | 00000 | 0100011
32 bit: 0000 0000 1101 0111 1010 0000 0010 0011
Kod: 0x00d7a023   Bellek: 23 a0 d7 00   objdump: aynı
```

#9 0x1030O addi a5,a5,1 (I)

```
Alanlar: 000000000001 | 01111 | 000 | 01111 | 0010011
32 bit: 0000 0000 0001 0111 1000 0111 1001 0011
Kod: 0x00178793   Bellek: 93 87 17 00   objdump: aynı
```

#10 0x10310 bge a5,a4,0x10288 (B)

```
Alanlar: 1 | 111011 | 01110 | 01111 | 101 | 1100 | 1 | 1100011
32 bit: 1111 0110 1110 0111 1101 1100 1110 0011
Kod: 0xf6e7dce3   Bellek: e3 dc e7 f6   objdump: aynı
```

Zorunlu satırlar: SID #1-2; koşullu branch #10; load #3; store #8; çağrı #4.

Ek F - M2 / 2c objdump doğrulama görüntüsü

10 elle kodlanan komutun adresi, machine word ve assembly karşılığı.

```
❯ cd /home/yund/comporg/cmpe3209-proje
  rg '10270:|10274:|10288:|1028c:|102a4:|102d0:|102d4:|102d8:|10
300:|10310:' objdump_O0.txt
135:   10270:   0012f7b7                lui     a5,0x12f
136:   10274:   bd178793                addi    a5,a5,-1071 # 12
ebd1 <__BSS_END__+0x11b265>
141:   10288:   fec42503                lw      a0,-20(s0)
142:   1028c:   f89ff0ef                jal     10214 <next_x>
148:   102a4:   02e7b733                mulhu   a4,a5,a4
159:   102d0:   00279793                slli    a5,a5,0x2
160:   102d4:   00f707b3                add     a5,a4,a5
161:   102d8:   00d7a023                sw      a3,0(a5)
171:   10300:   00178793                addi    a5,a5,1
175:   10310:   f6e7dce3                bge     a5,a4,10288 <mai
n+0x28>
```

Dosya: screenshots/objdump_m2_verify.png

Ek G - M2 YZ kullanım beyanı

Araç ve sürüm: Codex (GPT-6)

Adım: M2 - Assembly ve machine code

Ne için kullanıldı:

YZ aracı; RISC-V komut formatlarını ve bit alanlarının yerleşimini açıklamak, C–assembly eşleştirmesi ile -O0/-O1 karşılaştırmasına yardımcı olmak için kullanıldı.

Örnek istemler:

“-O0 derlemesinde C kodundaki ilk döngünün assembly karşılığını nasıl bulurum?” ve “bge neydi?”

Çıktıyı nasıl doğruladım:

Elle kodladığım 10 komutu objdump_O0.txt ile karşılaştırdım; -O0/-O1 sayımlarını objdump_O0.txt ve objdump_O1.txt üzerinden kontrol ettim. Tutanak sayfalarını görsel olarak inceledim.